

CI/CD-Pipelines am Beispiel von GitLab

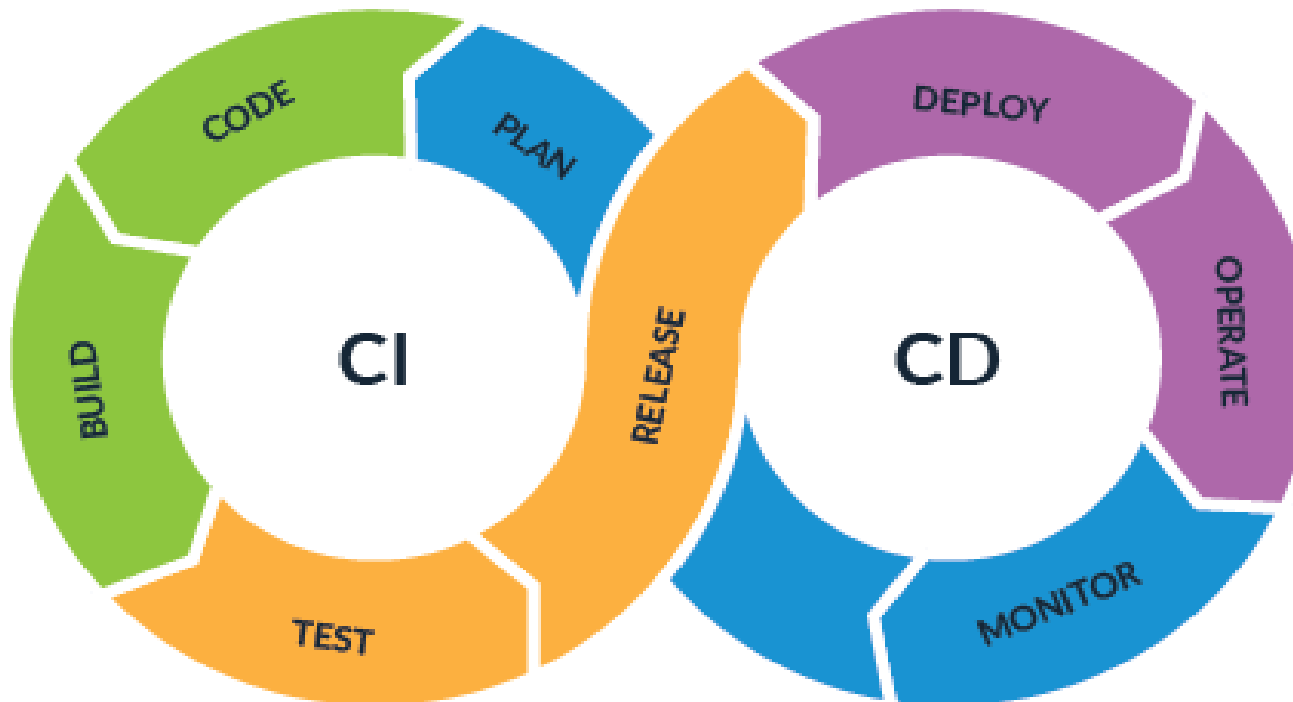
Container mit Schwerpunkt
Kubernetes



- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

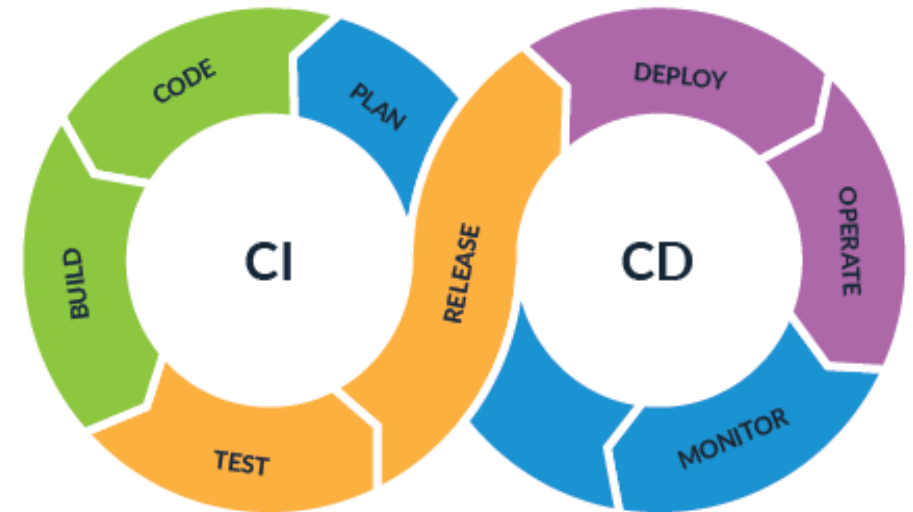
CI/CD steht für Continuous Integration / Continuous Delivery oder Continuous Deployment

- CI/CD ist eine Entwicklungs-Methode.
- CI/CD lässt sich folgende Phasen einteilen.



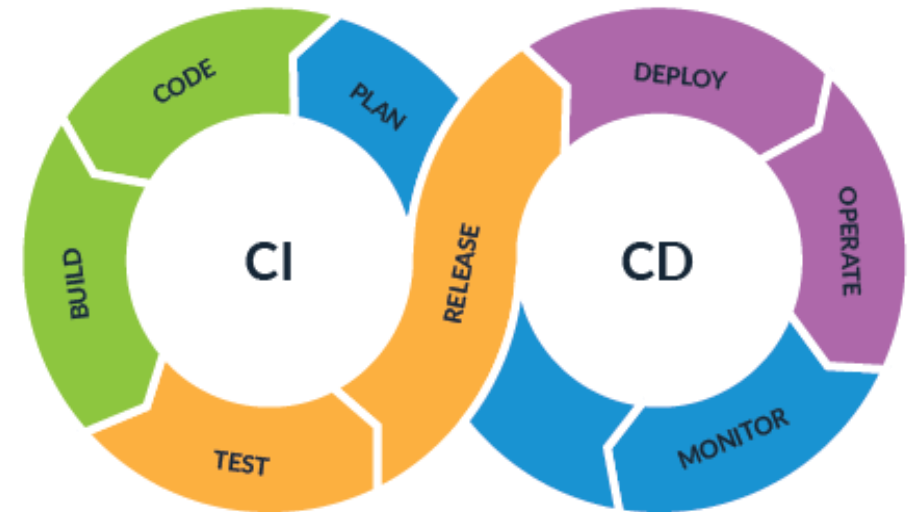
CI/CD - Überblick - Phase Plan

- Zusammentragen von Informationen: funktionale und nicht-funktionale Anforderungen.
- Strukturierung der Informationen: Tickets werden erstellt, Feature, Epic, Story, Task.
- Details der Anforderung dokumentieren: beispielsweise in einem Wiki.



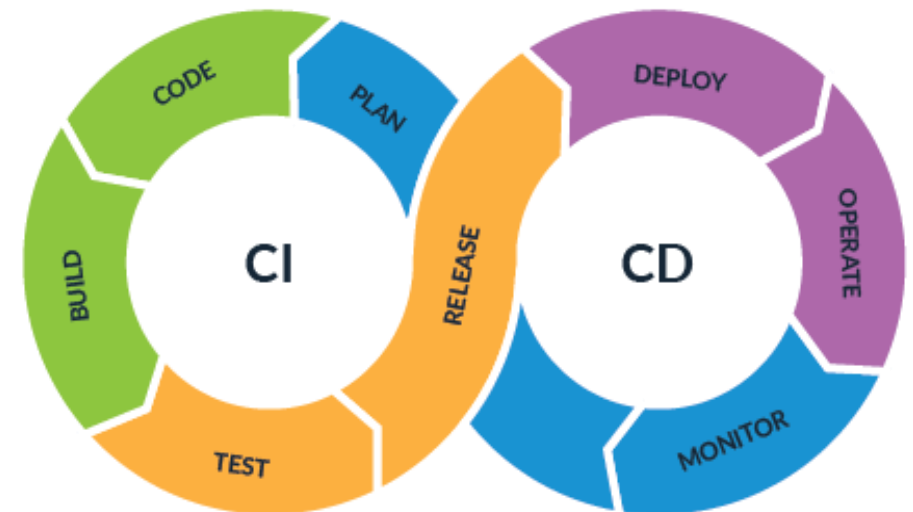
CI/CD - Überblick - Phase Code

- Das Entwickler-Team setzt Anforderung um.
- Der Code wird in einem Code-Repository (git) verwaltet.
- Entwicklung erfolgt in Branches - so ist eine parallele Arbeit an der Software möglich.



CI/CD - Überblick - Phase Build

- Aufgrund des Codes im Code-Repository findet der Build-Prozess statt.
- Der Build-Prozess ist eine erste objektive Überprüfung der Qualität des Codes.
 - Lässt sich der Code kompilieren?
 - Sind die Entwickler-Tests erfolgreich?
- Das Ergebnis des Build-Prozesses ist ein Software-Artefakt, das zur Ausführung gestartet werden kann.
- Der Build-Prozess kann das Software-Artefakt veröffentlichen: Das Software-Artefakt wird dazu in einem Repository (z. B. Nexus-Repository, Container-Repository).

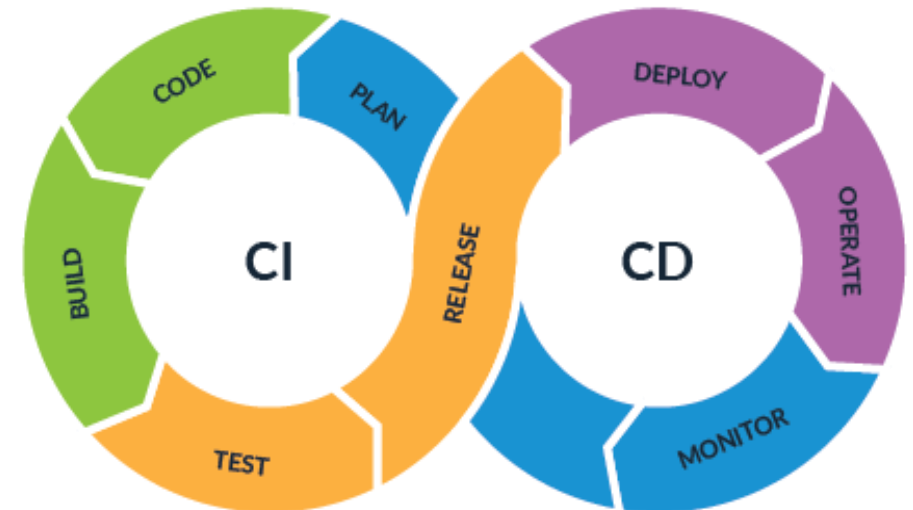


■ Test

- Das veröffentlichte Software-Artefakt wird in einer Test-Umgebung installiert.
- Tests überprüfen die funktionalen und nicht-funktionalen Anforderungen.
- Abweichungen vom erwarteten Ergebnis werden als Fehler (Bug) festgehalten.
- Bugs können zurück zu Code und Build führen.

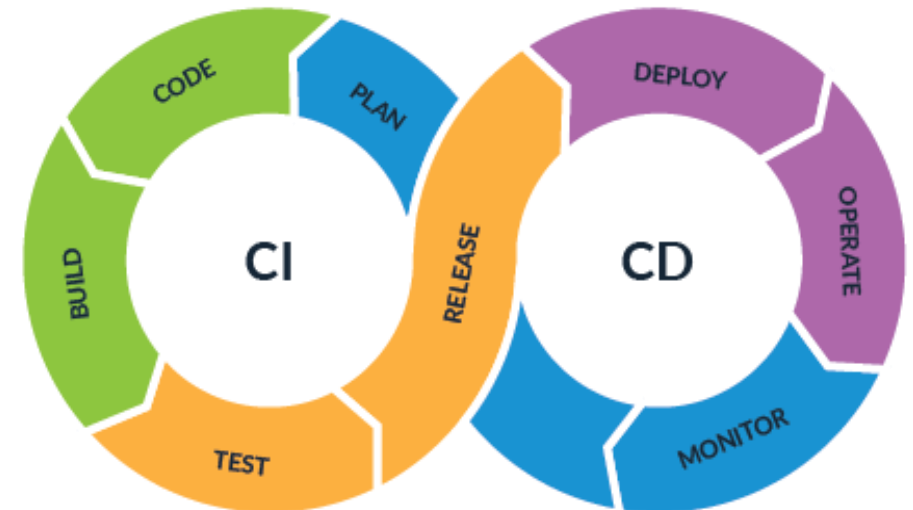
■ Release:

- In dieser Phase wird entschieden, dass das Software-Artefakt für den produktiven Betrieb geeignet ist.



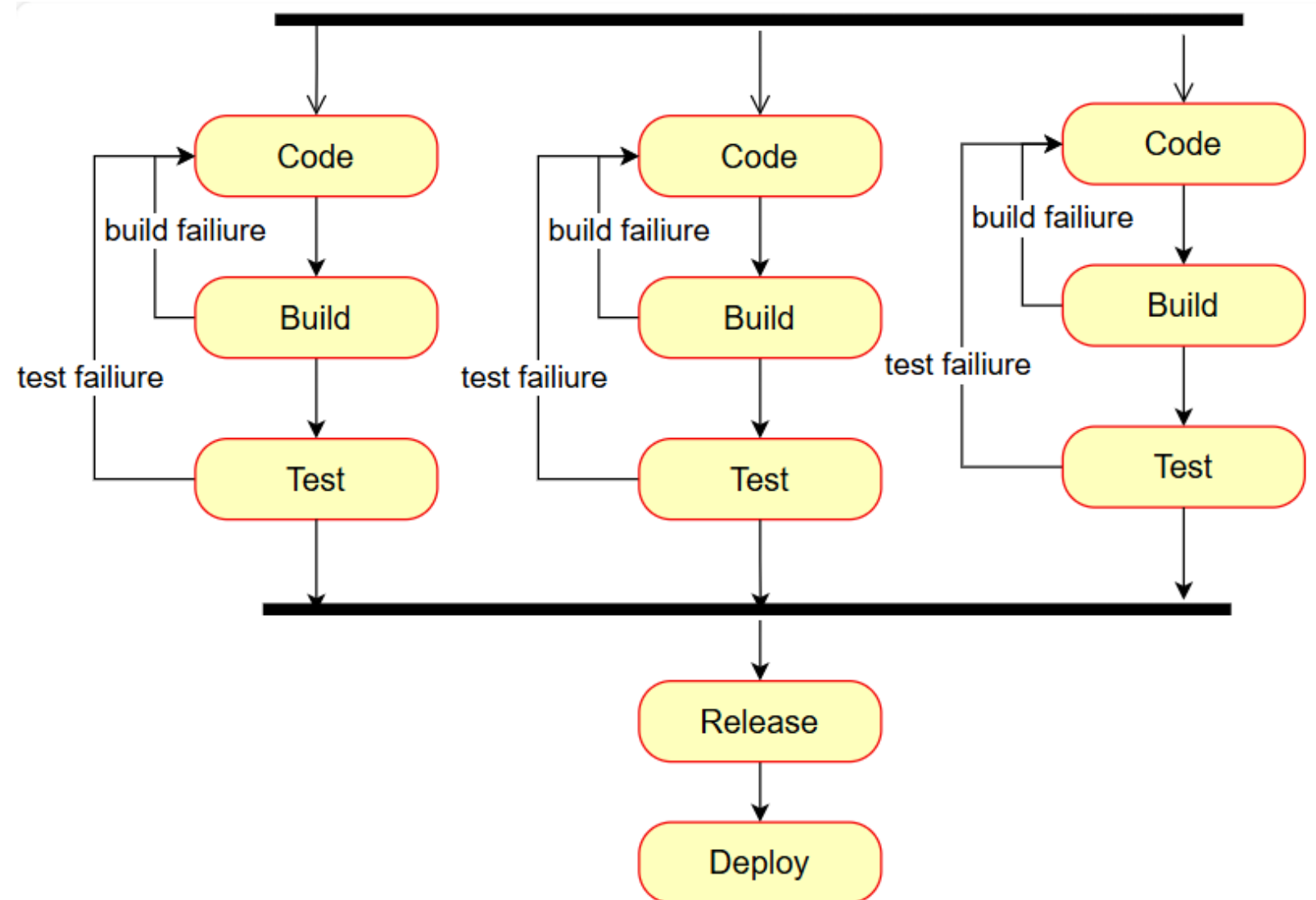
CI/CD - Überblick - Phasen Deploy, Operate, Monitor

- **Deploy**
 - Das Software-Artefakt wird in der Produktions-Umgebung in Betrieb genommen.
- **Operate**
 - Administratoren sorgen für einen (möglichst) störungsfreien Betrieb.
- **Monitor**
 - Wesentliche Eigenschaften der Anwendung werden überwacht: Beispiel: Last, Antwortzeiten, Verfügbarkeiten.



Text

- CI/CD folgt keinem Wasserfall-Modell.
- CI/CD erfolgt in der Realität iterativ: in vielen Zyklen von Code, Build und Test wird das Ziel erreicht.
- Build und Test werden vielfach aufgerufen.
- Ein hohes Maß an Automatisierung ist erforderlich.



- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

Ziele und Methoden

Ziele

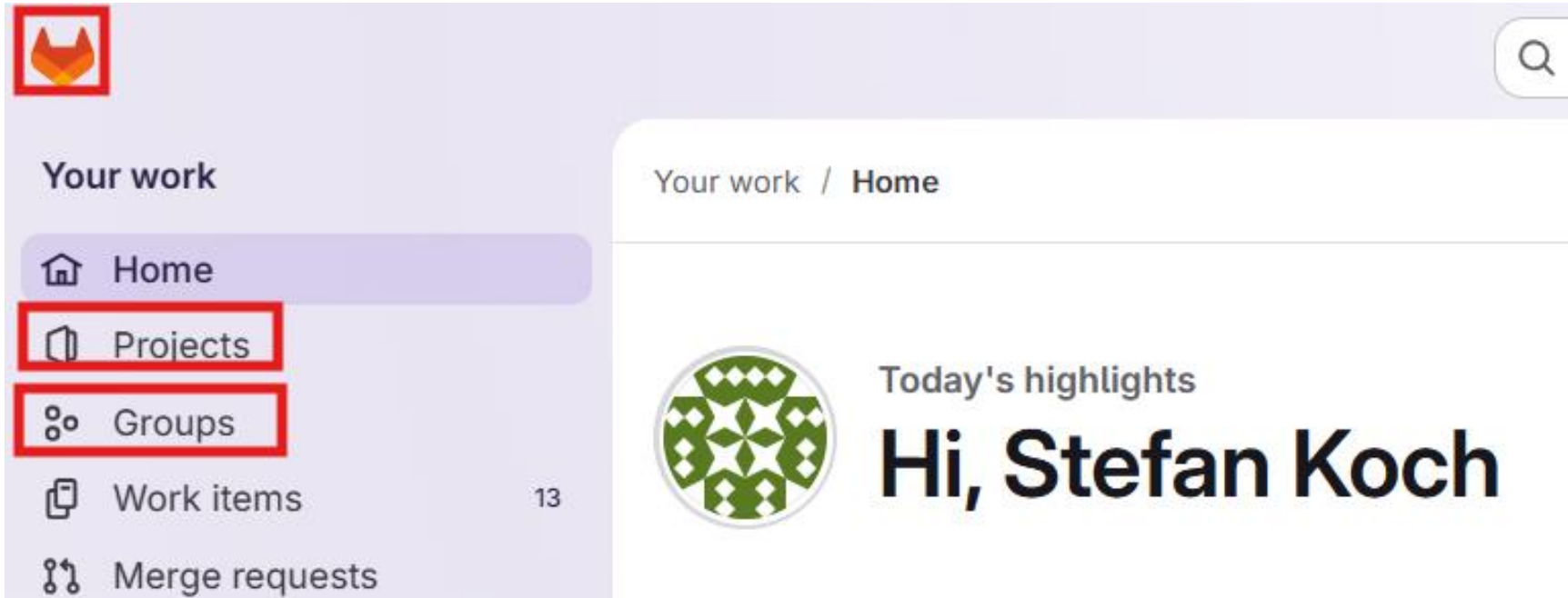
- CI/CD mit GitLab kennen lernen.
- Fokus auf den Build-Prozess mit
 - Delivery und Deployment
 - Zielumgebung Kubernetes

Methoden

- GitLab hat einen sehr großen Funktionsumfang.
- Exemplarisch werden die Prozesse anhand einer Beispiel-Anwendung vorgestellt.
- Handhabung der Werkzeuge wird eingeübt.

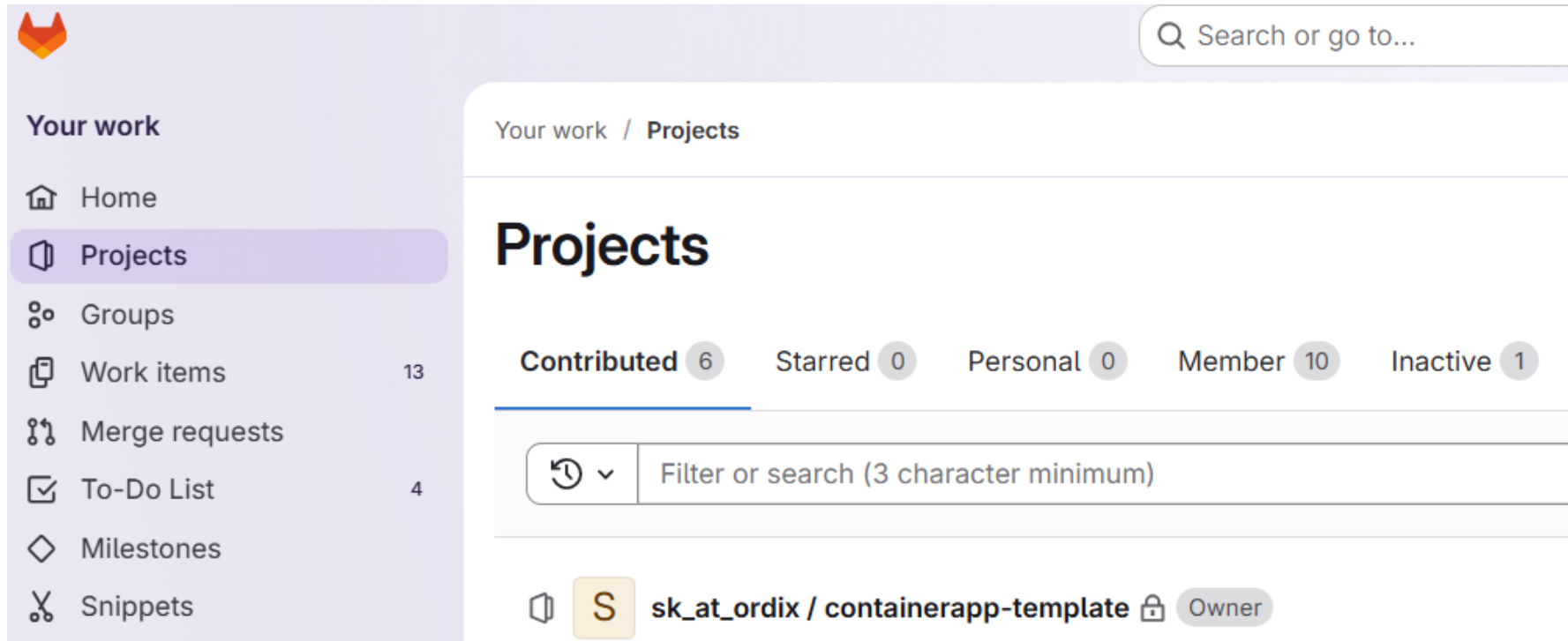
- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
 - Homepage
 - Projekte und Gruppen
 - Projekte forken
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

GitLab-Einführung - Homepage



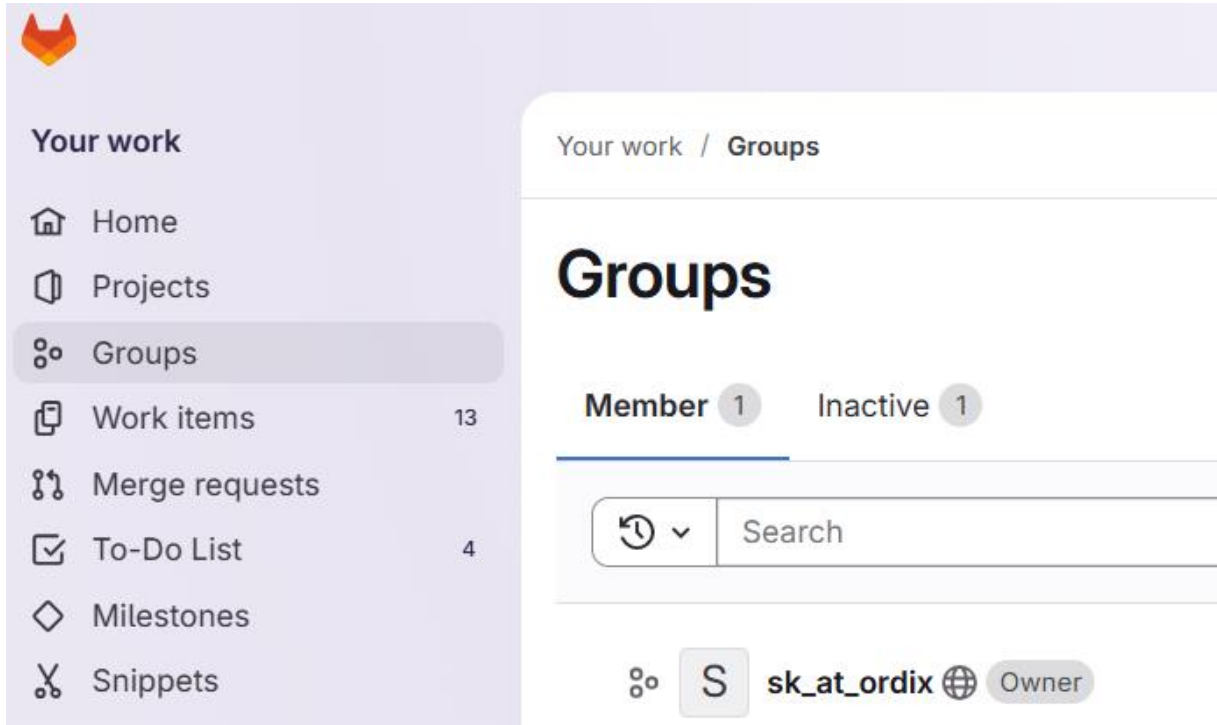
- Das Fuchs-Symbol ist ein Link zur Home-Page des Anwenders.
- Über die Homepage navigiert der Anwender zu
 - Projekten oder
 - Gruppen

GitLab-Einführung - Homepage > Projekts



- Der Anwender sucht nach Projekten.
- Verwendung von Standard-Filtern.
- Verwendung von Freitext-Suche.

GitLab-Einführung - Homepage > Groups



- Der Anwender sucht nach Gruppen.
- Verwendung von Standard-Filtern.
- Verwendung von Freitext-Suche.
- Gruppen können eine Hierarchie abbilden.

GitLab-Einführung - Homepage des Projekts

The screenshot shows the GitLab interface for a project named 'containerapp'. The left sidebar contains navigation links: Project, containerapp, Learn GitLab, Pinned, Work items (New), Merge requests, Manage, Plan, Code, Build, Secure, Deploy, Operate, Monitor, Analyze, and Settings. The main content area displays the project name 'containerapp' with a lock icon, a search bar, and a 'Search or go to...' input. Below this, there's a 'main' branch selector and a 'select branch' link. A merge request is shown: 'Merge branch 'authorize-for-group_sk_at_ordix' into 'main'' by Stefan Koch, authored 1 day ago. A table lists files and their last commit details:

Name	Last commit	Last update
.gitlab/agents	authenticate for group sk_at_ordix	1 day ago
src	version mit h2-Datenbank	3 weeks ago
.gitignore	Prototyp Kassenanwendung	4 months ago
README.md	Add some notes how to use	1 month ago
pom.xml	Kasse mit h2-Persistenz	4 months ago

On the right, the 'Project information' section shows: 10 Commits, 7 Branches, 0 Tags, and 260.5 MiB Project Storage. Below this, there are links to README, Enable Auto DevOps, Add Kubernetes cluster, Add Wiki, Configure Integrations, and Enable Observability.

Project-Sidebar

Git-Repository

Project-Infos

GitLab-Einführung - Forken eines Projekts (1)

- Forken: ein Projekt wird von mehreren Teams weiterentwickelt.
- Jedes Team verfügt danach über ein eigenes Projekt.
- **Startpunkt:** Homepage des Ursprungsprojekts.

sk_at_ordix / containerapp-template

The screenshot shows the GitLab interface for the project 'containerapp-template' by the user 'sk_at_ordix'. The project name is displayed with a lock icon, indicating it is private. Below the name, the 'main' branch is selected. To the right of the project name, there are buttons for 'Notifications', 'Stars' (0), and 'Fork' (0). The 'Fork' button is circled in red. Below these buttons, there are options to '+', 'Find file', and 'Code'. On the right side, there is a section titled 'Project information'.

GitLab-Einführung - Forken eines Projekts (2)

Project name

containerapp-user01

Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.

Project URL

https://gitlab.com/

sk_at_ordix



Project slug

containerapp-user-01

Want to organize several dependent projects under the same namespace? [Create a group](#)

Project description (optional)

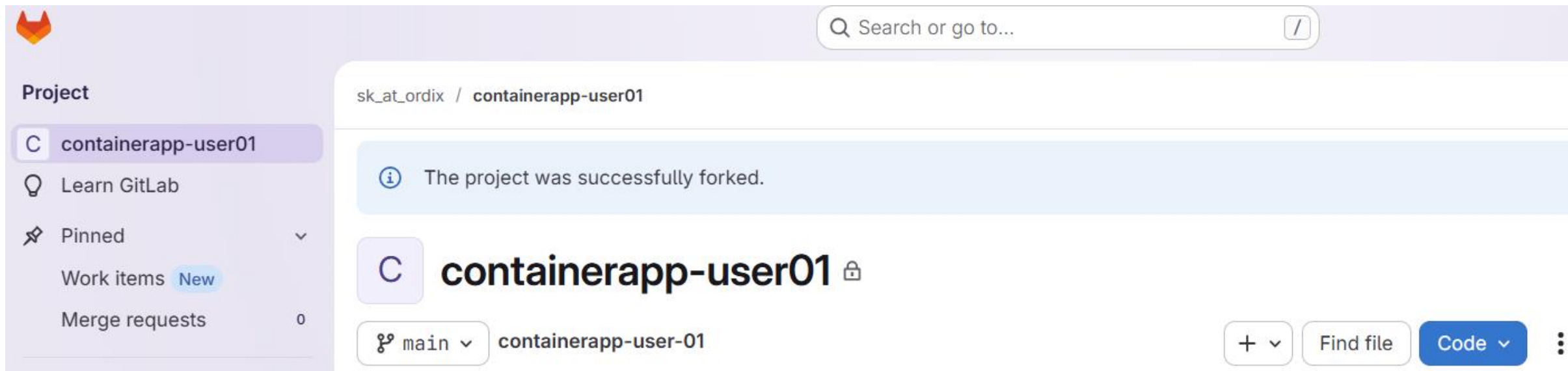
Branches to include

- ☐ All branches
- ☒ Only the default branch **main**

Visibility level

- ☒  Private
Project access must be granted explicitly to each user. If this project is part of a group, access will be granted to members of the group.
- ☐  Internal

GitLab-Einführung - Forken eines Projekts (3)



- Nach einiger Zeit navigiert GitLab zur Homepage des navigierten Projekts.

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
 - Zielsetzung
 - Komponenten für den Build-Prozess
 - Erstellung des Build-Skripts für die Bibliothek
 - Erstellung des Build-Skripts für das Container-Image
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
 - Zielsetzung
 - Komponenten für den Build-Prozess
 - Erstellung des Build-Skripts für die Bibliothek
 - Erstellung des Build-Skripts für das Container-Image
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

1. Erstellung eines ausführbaren Programms als Software-Artefakt.
 - Build-Skript *.gitlab-ci.yml* als Teil des Source-Codes erstellen.
2. Erstellung eines Container-Images für die Ausführung des Programms.
 - Container-Image in die Container-Registry pushen.
 - Programm im Kubernetes-Cluster ausführen.

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
 - Zielsetzung
 - Komponenten für den Build-Prozess
 - Erstellung des Build-Skripts für die Bibliothek
 - Erstellung des Build-Skripts für das Container-Image
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

Build-Server

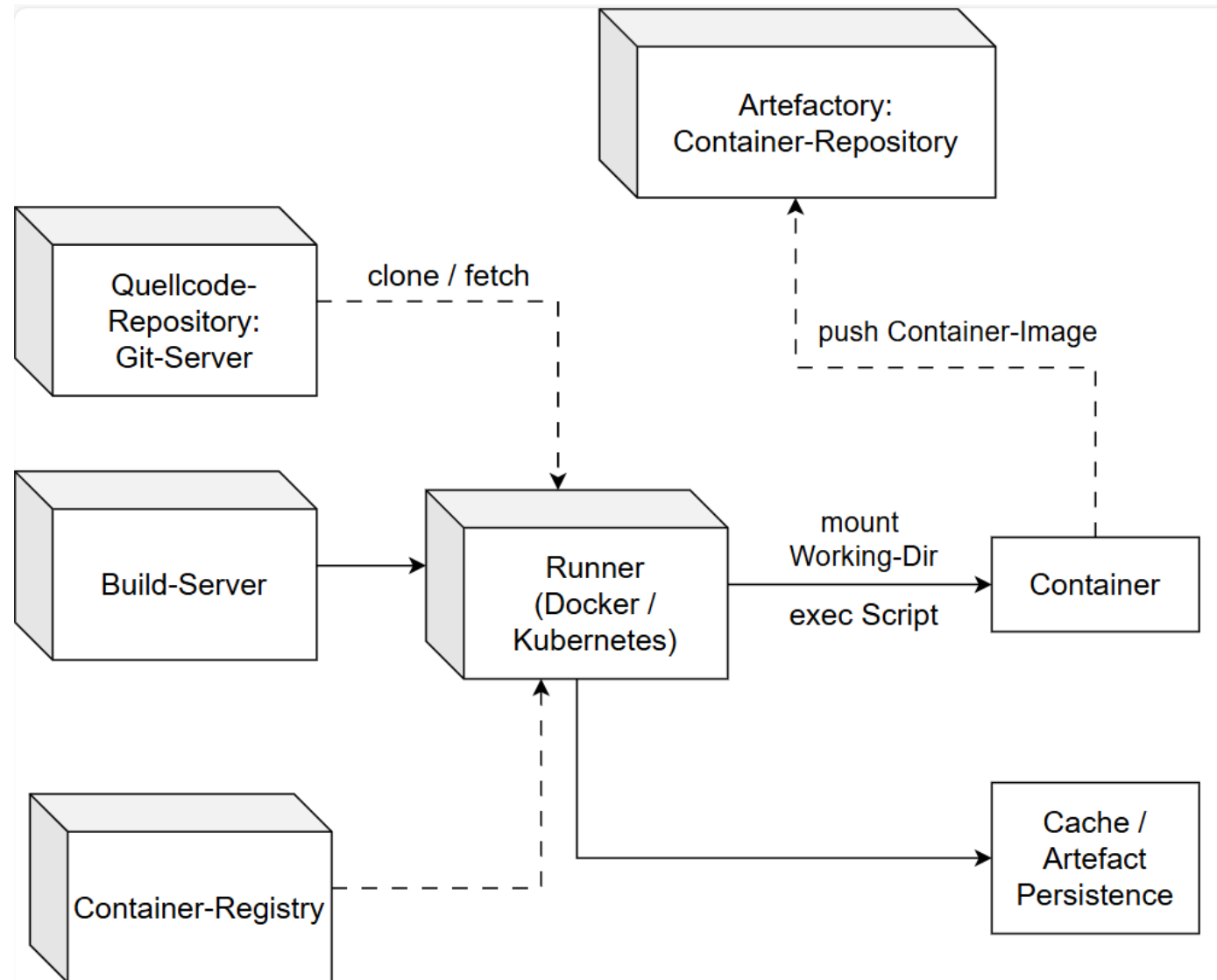
- Startet den Build-Prozess.
- Kommuniziert mit dem Runner

Runner

- Klont Quellcode-Branch.
- Erstellt Container.
- Steuert den Container: Führt das Build-Skript aus.

Artefactory

- Speichert das Software-Artefakt.
- Beispiel: Container-Repository speichert das Container-Image



- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
 - Zielsetzung
 - Komponenten für den Build-Prozess
 - Erstellung des Build-Skripts für die Bibliothek
 - Erstellung des Build-Skripts für das Container-Image
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

Continuous Delivery - Build-Skript erstellen (1)

The screenshot shows the GitLab web interface for a project named 'containerapp'. The interface includes a file explorer on the left, a commit history table, and a 'Code' dropdown menu on the right.

Files: .gitignore, README.md, pom.xml

Commit History:

Name	Last commit
src	version mit h2-Datenbank
.gitignore	Prototyp Kassenanwendung
README.md	Add some notes how to use

Code Menu:

- Clone with SSH: git@gitlab.com:sk_at_ordix/conta
- Clone with HTTPS: https://gitlab.com/sk_at_ordix/c
- Open with: Web IDE

- Über die Projekt-Homepage wählen Sie die Web IDE, zum Editieren des Source-Codes:
[Code] Web IDE
- WEB IDE ist eine komplette, web-basierte, Entwicklungsumgebung.

Continuous Delivery - Build-Skript erstellen (2)

- Projekt-Verzeichnis im Projekt-Explorer aufklappen.
- Root-Verzeichnis markieren.



- [Datei+] anklicken
- Dateiname **.gitlab-ci.yml** eintragen.



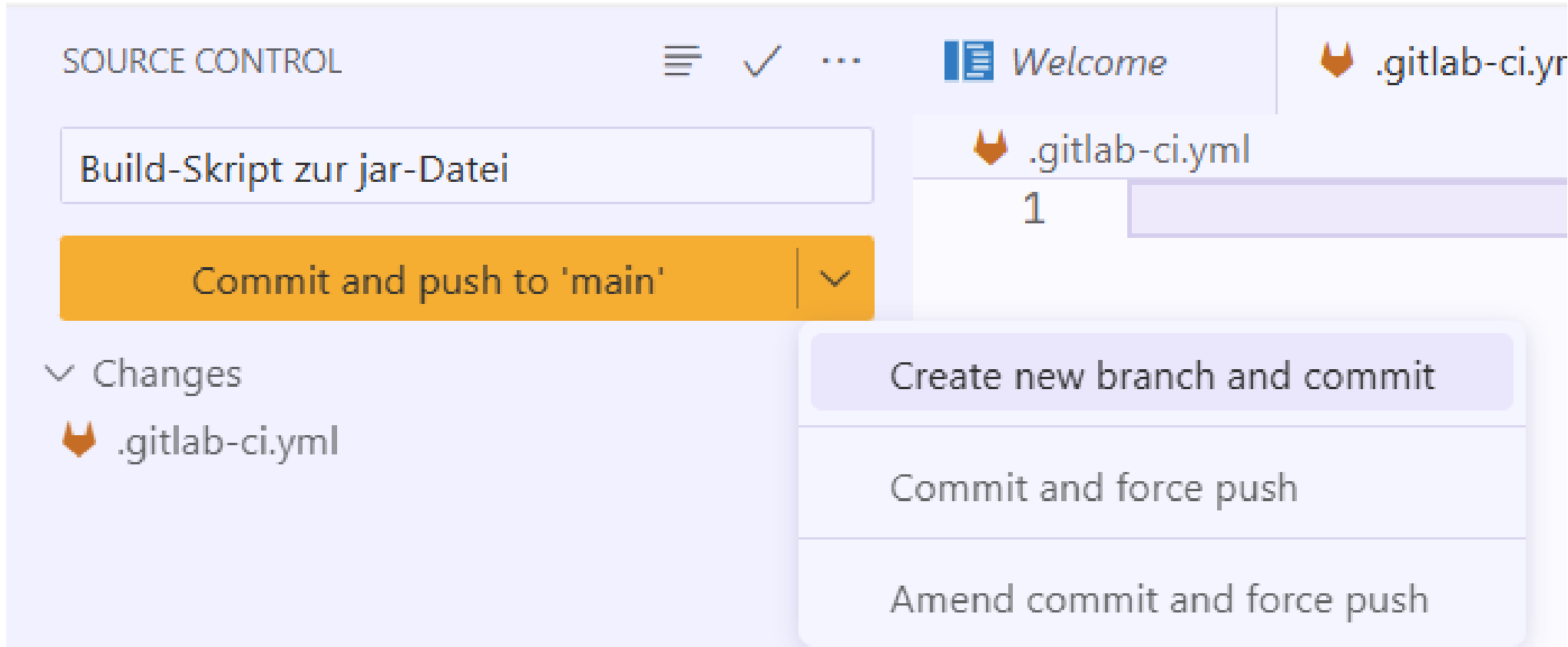
- **.gitlab-ci.yml** ist der Dateiname des Build-Skripts.
 - Der Name muss exakt so heißen.
 - Das Build-Skript muss im obersten Verzeichnis des Projektes liegen.
- Das Build-Skript wird zusammen mit dem Source-Code verwaltet.
- GitLab wertet das Build-Skript bei einer Veränderung (push) des Git-Repositories vom Projekt aus. Existiert das Build-Skript, so wird es automatisch ausgeführt.
- .gitlab-ci.yml wird im YAML-Format geschrieben.
- .gitlab-ci.yml verfügt über eine umfangreiche Syntax.

Continuous Delivery - Build-Skript erstellen (4)

```
image: ·maven:3.9.9-eclipse-temurin-21¶
¶
stages:¶
  ·- ·build¶
¶
build:¶
  ·stage: ·build¶
  ·script:¶
    ·- ·mvn ·clean ·package¶
  ·artifacts:¶
    ·paths:¶
      ·- ·target/* ·.jar¶
```

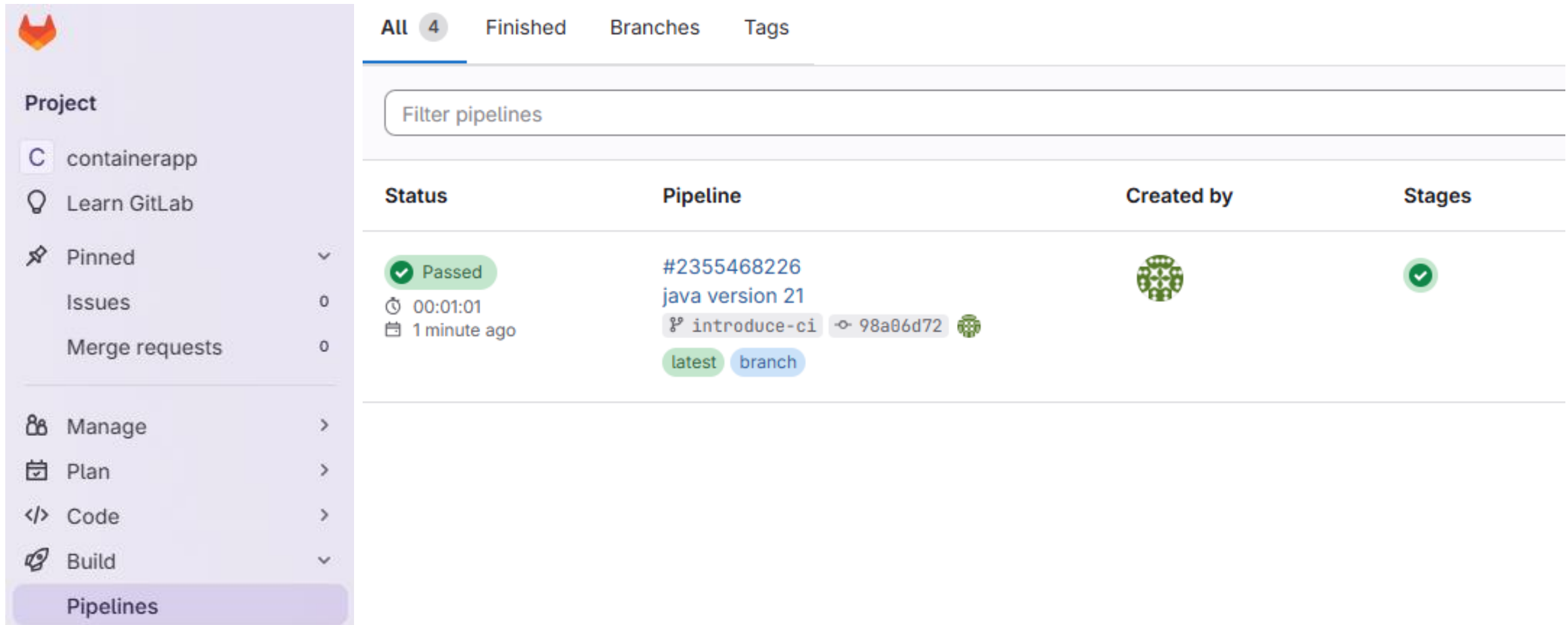
- Build-Skript .gitlab-ci.yml für die Erstellung einer jar-Datei.
- Einrückungen durch Leertasten sind wichtig!
- Leertasten werden im Listing oben durch einen Punkt sichtbar gemacht.

Continuous Delivery - Build-Skript erstellen (5)



- Abspeichern der Änderung in dem Branch *introduce-ci*.
- Nach dem Commit wird das Build-Skript direkt ausgeführt.

Continuous Delivery - Build-Skript ausführen

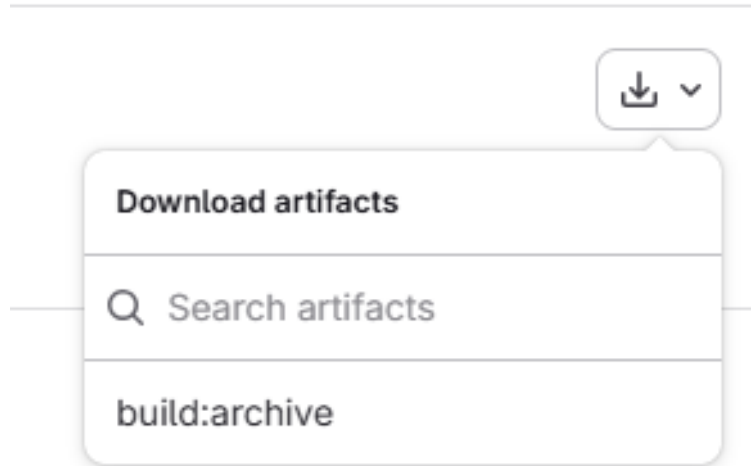


The screenshot displays the GitLab CI/CD interface. On the left is a sidebar with the 'Project' menu open, showing options like 'containerapp', 'Learn GitLab', 'Pinned', 'Issues', 'Merge requests', 'Manage', 'Plan', 'Code', 'Build', and 'Pipelines' (which is highlighted). The main area shows a list of pipelines under the 'All' tab, which has a count of 4. A search bar labeled 'Filter pipelines' is present. The pipeline list has columns for 'Status', 'Pipeline', 'Created by', and 'Stages'. One pipeline is shown with a 'Passed' status (green checkmark), ID '#2355468226', and a duration of '00:01:01'. The pipeline name is 'java version 21' and it includes a stage 'introduce-ci' with a commit hash '98a06d72'. The 'Created by' column shows a user icon, and the 'Stages' column shows a green checkmark.

Status	Pipeline	Created by	Stages
✓ Passed ⌚ 00:01:01 📅 1 minute ago	#2355468226 java version 21 🔗 introduce-ci 98a06d72 latest branch		✓

- In der Projekt-Homepage wählen Sie Build > Pipeline.
- Wenn alles klappt wird die Pipeline grün.

Actions



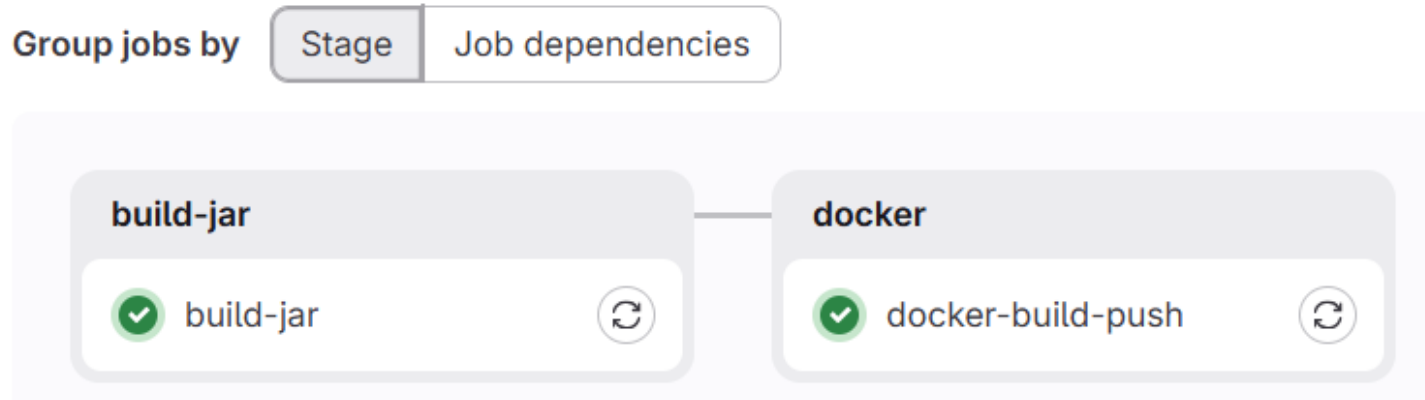
```
$ $JAVA_HOME/bin/java -jar kasse-0.1.0-SNAPSHOT.jar
```

```
$ curl -X POST http://localhost:8080/api/v1/kasse/K1 \
-H "Content-Type: application/json" \
-d '{"bonItem": 1234}'
{"id":2,"positionen":[{"id":2,"bonItem":1234}],"kassenId":
:"K1","status":"IN_PROGRESS","summe":1234}
```

- Das Software-Artefakt laden Sie im Pipeline-Dialog herunter: Download artefacts > build:archive
- Das Software-Artefakt probieren Sie mit den obigen Kommandos aus.
 - Starten der Anwendung
 - REST-Request absetzen

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
 - Zielsetzung
 - Komponenten für den Build-Prozess
 - Erstellung des Build-Skripts für die Bibliothek
 - Erstellung des Build-Skripts für das Container-Image
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example

Continuous Delivery - Container-Image erstellen (1)



- Das Build-Skript besteht nun aus 2 Stages
 - build-jar: für das Erstellen der jar-Datei mit dem Job build-jar.
 - docker: für das Erstellen und Pushen des Container-Images mit dem Job docker-build-push.

Continuous Delivery - Container-Image erstellen (2)

Das Dockerfile speichern Sie im Root-Verzeichnis des Projektes.

- Basis-Image ist eclipse-temurin:21-jre
Vollständiger Name für buildah erforderlich.
- Arbeitsverzeichnis ist /app
- Die jar-Datei aus dem Verzeichnis target wird als app.jar in das Arbeitsverzeichnis kopiert.
- Das Start-Kommando ist
java -jar /app/app.jar

```
FROM docker.io/library/eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Continuous Delivery - Container-Image erstellen (3)

```
image: ·maven:3.9.9-eclipse-temurin-21¶
¶
stages:¶
  ··- ·build-jar¶
  ··- ·docker¶
¶
build-jar:¶
  ··stage: ·build-jar¶
|··script:¶
  ····- ·mvn ·clean ·package¶
  ··artifacts:¶
  ···name: ·"jar-artifact"¶
  ···paths:¶
  ·····- ·target/*.jar¶
¶
```

Erster Teil des Build-Skripts:

- Definition des Stages
- Erzeugen der jar-Datei
- Im zweiten Teil wird das Docker-Image erzeugt...

Continuous Delivery - Container-Image erstellen (4)

Zweiter Teil des Build-Skripts erzeugt und schreibt das Container-Image in die Container-Registry von GitLab.

- *image*: quay.io/buildah/stable:latest
- *needs*: Abhängigkeit zum 1. Job
 - Das Artefakt des 1. Jobs wird benötigt.
- *script*: Besteht aus 3 Befehlen.
 - buildah login: Einloggen bei der Registry
 - buildah bud: Erzeugen vom Container-Image
 - buildah push: Container-Image pushen

```
docker-build-push:
  stage: docker
  image:
    name: quay.io/buildah/stable:latest
    entrypoint: [""]
  variables:
    IMAGE_NAME: "$CI_REGISTRY_IMAGE"
    IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"
    STORAGE_DRIVER: "vfs"
    BUILDDAH_ISOLATION: "chroot"
  needs:
    - job: build-jar
      artifacts: true
  script:
    - buildah login
      --username "$CI_REGISTRY_USER"
      --password "$CI_REGISTRY_PASSWORD"
      "$CI_REGISTRY"
    - buildah bud
      --storage-driver "$STORAGE_DRIVER"
      --isolation "$BUILDDAH_ISOLATION"
      --format docker
      --security-opt seccomp=unconfined
      --file "$CI_PROJECT_DIR/Dockerfile"
      --tag "$IMAGE_NAME:$IMAGE_TAG"
      --tag "$IMAGE_NAME:latest"
      "$CI_PROJECT_DIR"
    - buildah push
      --storage-driver "$STORAGE_DRIVER"
      --retry 6
      --retry-delay 10s
      "$IMAGE_NAME:latest"
```

Continuous Delivery - Container-Image erstellen (5)

sk_at_ordix / containerapp-user01 / Container registry

Container registry

1 Image repository ⌚ Cleanup is not scheduled. [Set up cleanup](#)

Filter results

... containerapp-user-01 

2 tags

- Nach der Ausführung der Pipeline ist ein Container-Image in der Container-Registry sichtbar.
- Die URL für das Image ist über das Kopier-Symbol erreichbar.

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
 - Test-Installation als Pod
 - Veröffentlichung des Microservices
- Continuous Deployment by Example

Manuelles Deployment - Test-Installation als POD - Secret

```
kubectl · create · secret · docker-registry · gitlab-registry · \¶  
· · --docker-server=<URL · des · Servers> · \¶  
· · --docker-username=<meine-Mail-Adresse> · \¶  
· · --docker-password=<mein-Kennwort> · \¶  
· · --dry-run=client · -o · yaml · > · gitlab-registry-secret.yaml¶
```

```
$ · kubectl · apply · -f · gitlab-registry-secret.yaml¶  
secret/gitlab-registry · created¶  
$ · kubectl · get · secrets¶  
NAME · . . . . . TYPE · . . . . . DATA · . . . AGE¶  
gitlab-registry · . . . kubernetes.io/dockerconfigjson · . . . 1 · . . . 7s¶
```

- Zielsetzung: Überprüfung des Container-Images
- Vorgehen:
 - Secret für den Zugriff auf die Container-Registry erstellen
 - POD auf Grundlage des Container-Images erstellen und testen

Manuelles Deployment - Test-Installation als POD - POD Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: containerapp
spec:
  imagePullSecrets:
    - name: gitlab-registry
  containers:
    - name: containerapp
      image: <URL des Image>
      imagePullPolicy: Always
      ports:
        - containerPort: 8080
```

- Secret ist für das Container-Image zu referenzieren.
- URL des Container-Images ist einzusetzen.

Manuelles Deployment - Test-Installation als POD - POD testen

```
$ kubectl exec -it containerapp -- /bin/bash
# curl -X POST http://localhost:8080/api/v1/kasse/K1 \
-H "Content-Type: application/json" \
-d '{"bonItem": 1234}'
{"id":2,"positionen":[{"id":2,"bonItem":1234}], "kassenId":"K1", "status":"IN
_PROGRESS", "summe":1234}
# curl http://localhost:8080/api/v1/kasse/K1
{"id":2,"positionen":[{"id":2,"bonItem":1234}], "kassenId":"K1", "status":"FI
NALIZED", "summe":1234}root@containerapp:/app# exit
```

- Der Test erfolgt über den gestarteten Container
- Im Test wird eine Bon-Position erstellt (POST).
- Danach abkassiert (GET).

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
 - Test-Installation als Pod
 - Veröffentlichung des Microservices
- Continuous Deployment by Example

Veröffentlichung - Deployment

- Secret ist für das Container-Image zu referenzieren.
- URL des Container-Images ist einzusetzen.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: containerapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: containerapp
  template:
    metadata:
      labels:
        app: containerapp
    spec:
      imagePullSecrets:
        - name: gitlab-registry
      containers:
        - name: containerapp
          image: <URL des Image>
          ports:
            - containerPort: 8080
```

Veröffentlichung - Service

- URL des Service wird durch den Namen angegeben.
- Service innerhalb des Kubernetes-Clusters testen.

```
apiVersion: v1
kind: Service
metadata:
  name: containerapp
spec:
  selector:
    app: containerapp
  ports:
    - port: 80
      targetPort: 8080
    type: ClusterIP
```

```
$ kubectl run curltest \
  --rm --it \
  --image=curlimages/curl \
  --restart=Never \
  -- sh
~$ curl http://containerapp/api/v1/kasse
[{"id":1,"positionen":[{"id":1,"bonItem":4711}], "kassenId":"Test", "status":
"IN_PROGRESS", "summe":4711}]
```

Veröffentlichung - Ingress

- *host*: an Umgebung anpassen.
- *path*: an Umgebung anpassen.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: containerapp
  annotations:
    nginx.ingress.kubernetes.io/use-regex: "true"
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
    - host: microk8s
      http:
        paths:
          - path: /containerapp-userXXX(/|$)(.*)
            pathType: Prefix
            backend:
              service:
                name: containerapp
                port:
                  number: 80

```

Veröffentlichung - Ingress testen

- Ausgabe zum erstellten Ingress:

```
$ kubectl get ingress
NAME .....CLASS .....HOSTS .....ADDRESS .....PORTS .....AGE
containerapp .....nginx .....microk8s .....127.0.0.1 .....80 .....39s
```

- Der Test erfolgt über den Host-Namen und den Path.
- Beispiel:

```
$ curl -H "Host: microk8s" http://microk8s/containerapp/api/v1/kasse
[{"id":1,"positionen":[{"id":1,"bonItem":4711}], "kassenId":"Test", "status":
"IN PROGRESS", "summe":4711}]
```

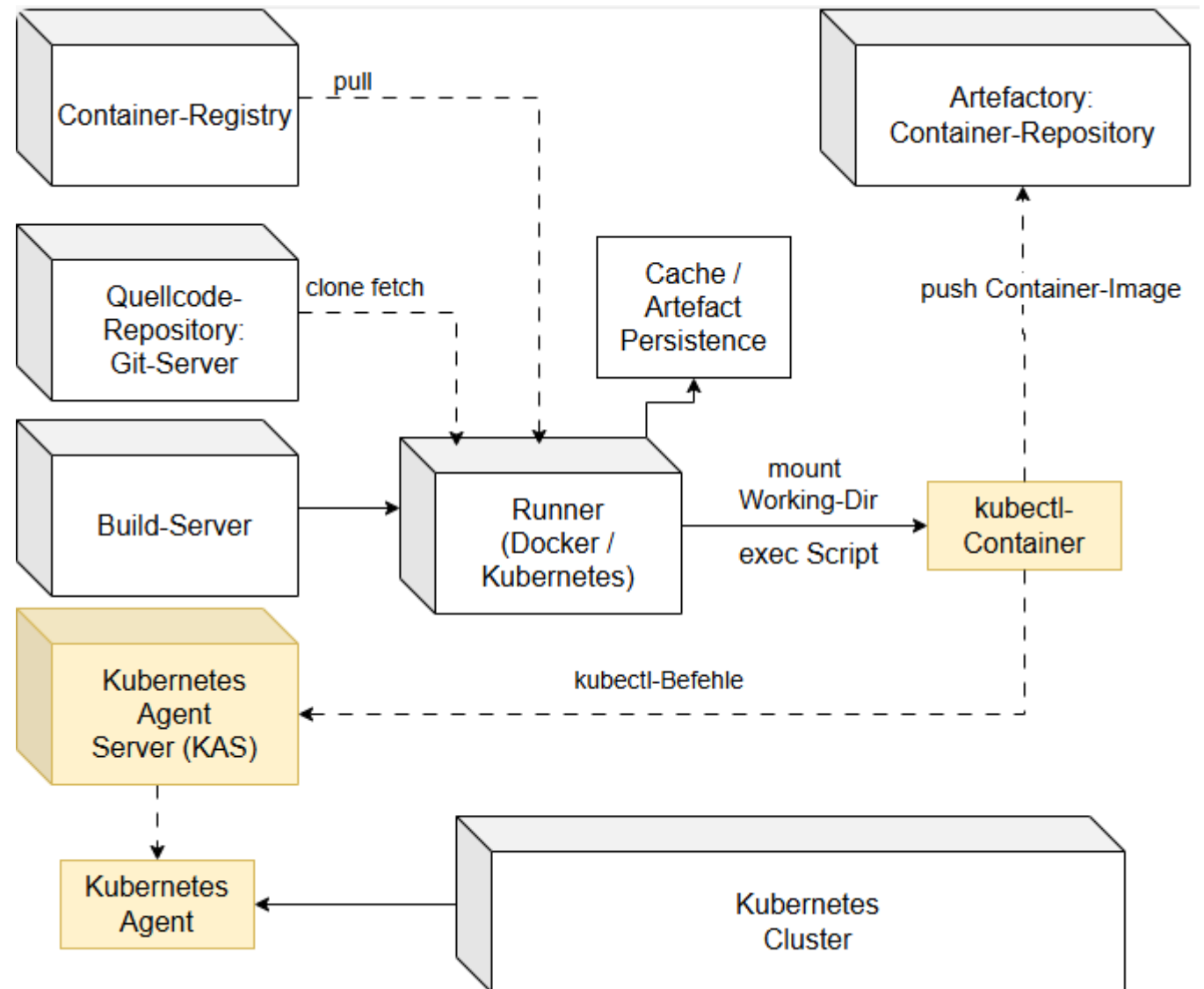
- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example
 - Context GitLab Kubernetes Agent Server
 - Automatisches Deployment

Agenda

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example
 - Context GitLab Kubernetes Agent Server
 - Automatisches Deployment

Kubernetes Agent Server (KAS)

- Kubernetes Agent Server ist Bestandteil von GitLab.
- Kubernetes Agent wird auf dem Kubernetes Cluster installiert.
- Befehle werden
 - über den Runner
 - an den kubectl-Container
 - über den Kubernetes Agent Server
 - an den Kubernetes Agent überliefert
 - und im Kubernetes Cluster ausgeführt.



Kubernetes Agent - Autorisieren von Projekten

- Ein Projekt definiert
 - Agenten-Record: Verbindung zum Kubernetes Agenten.
 - Autorisiert Projekte über `.gitlab/agents/<Agent>/config.yaml`.
 - *Agent*: Name des Agenten-Records.
- Autorisierung kann erfolgen
 - *projects*: autorisierte Projekte werden als Liste angegeben.
 - *groups*: autorisierte Gruppen werden als Liste angegeben.

```
ci_access:[]  
  ··projects:[]  
  ····-·id:·sk_at_ordix/containerapp[]
```

```
ci_access:[]  
  ··groups:[]  
  ····-·id:·seminar[]
```

Kubernetes Agent - Testen

- Verwendet wird ein Image, das kubectl-Befehle ausführen kann.
- Damit der KAS mit dem Agenten-Record ausgewählt wird, wird KUBE_CONTEXT definiert.
- Autorisierung kann erfolgen
 - *projects*: autorisierte Projekte werden als Liste angegeben.
 - *groups*: autorisierte Gruppen werden als Liste angegeben.
- GitLab erstellt eine kubeconfig, so dass kubectl-Befehle an den KAS gehen.

```
stages:
  - deploy
deploy:
  stage: deploy
  image:
    name: bitnami/kubectl:latest
    entrypoint: [""]
  variables:
    KUBE_CONTEXT: <Projekt mit Agenten-Record>:<Agent>
  before_script:
    - kubectl config use-context "$KUBE_CONTEXT"
  script:
    - kubectl get ns
    - kubectl get pod
```

Agenda

- CI/CD Überblick
- Ziele und Methode
- Einführung in GitLab
- Continuous Delivery by Example
- Manuelles Deployment in Kubernetes
- Continuous Deployment by Example
 - Context GitLab Kubernetes Agent Server
 - Automatisches Deployment

Automatisches Deployment - Projekt-Struktur

▼ k8s

 deployment.yaml

 ingress.yaml

 service.yaml

> src

 .gitignore

 .gitlab-ci.yml

 Dockerfile

 README.md

 pom.xml

- Dateien / Verzeichnisse im Projekt
- k8s: Verzeichnis für Manifest-Dateien
- .gitlab-ci.yml: Build-Skript
- Dockerfile: Für das Image-Build.

Automatisches Deployment - Build-Skript (1)

- Teil 1
- Allgemeine Informationen
- Bauen des jar-Files.
- Variable KUBE-CONTEXT einsetzen.
- KUBE_NAMESPACE anpassen.

```
image: ·maven:3.9.9-eclipse-temurin-21¶
¶
stages:¶
  ·- ·build¶
  ·- ·docker¶
  ·- ·deploy¶
¶
variables:¶
  ··IMAGE_NAME: ·"$SCI_REGISTRY_IMAGE"¶
  ··IMAGE_TAG: ·"$SCI_COMMIT_SHORT_SHA"¶
  ··KUBE_CONTEXT: ·"<Projekt ·mit ·Agenten-Record>: <Agent>"¶
  ··KUBE_NAMESPACE: ·"dev-namespace"¶
¶
build-jar:¶
  ··stage: ·build¶
  ··script:¶
    ···- ·mvn ·clean ·package¶
  ··artifacts:¶
    ···name: ·"jar-artifact"¶
    ···paths:¶
      ·····- ·target/*.jar¶
  ¶
```

Automatisches Deployment - Build-Skript (2)

- Teil 2
- Mit buildah:
 - authentisieren
 - Container-Image erstellen
 - Image pushen

```

docker-build-push:
  stage: docker
  image:
    name: quay.io/buildah/stable:latest
    entrypoint: [""]
  variables:
    IMAGE_NAME: "$CI_REGISTRY_IMAGE"
    IMAGE_TAG: "$CI_COMMIT_SHORT_SHA"
    STORAGE_DRIVER: "vfs"
    BUILDDAH_ISOLATION: "chroot"
  needs:
    - job: build-jar
      artifacts: true
  script:
    - buildah login
      --username "$CI_REGISTRY_USER"
      --password "$CI_REGISTRY_PASSWORD"
      "$CI_REGISTRY"
    - buildah bud
      --storage-driver "$STORAGE_DRIVER"
      --isolation "$BUILDDAH_ISOLATION"
      --format docker
      --security-opt seccomp=unconfined
      --file "$CI_PROJECT_DIR/Dockerfile"
      --tag "$IMAGE_NAME:$IMAGE_TAG"
      --tag "$IMAGE_NAME:latest"
      "$CI_PROJECT_DIR"
    - buildah push
      --storage-driver "$STORAGE_DRIVER"
      --retry 6
      --retry-delay 10s
      "$IMAGE_NAME:latest"

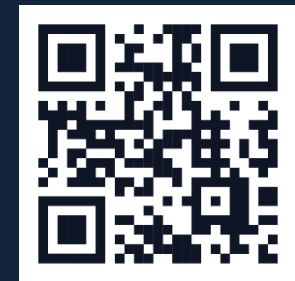
```

Automatisches Deployment - Build-Skript (3)

- Teil 3
- Automatisches Deployment
- *create namespace*: Namensraum wird angelegt, wenn es ihn noch nicht gibt.
- Test wie bei dem manuellen Deployment.

```
deploy:␣  
  ·stage:·deploy␣  
  ·tags:␣  
  ···-·onkubernetes␣  
  ·image:␣  
  ···name:·bitnami/kubectl:latest␣  
  ···entrypoint:·[""]␣  
  ·needs:␣  
  ···-·job:·docker-build-push·␣  
  ·script:␣  
  ···-·kubectl·config·use-context·"$KUBE_CONTEXT"␣  
  ···-·kubectl·create·namespace·"$KUBE_NAMESPACE"·--dry-run=client·--o·yaml·|·  
  kubectl·apply·-f·-␣  
  ···-·kubectl·-n·"$KUBE_NAMESPACE"·apply·-f·k8s/␣  
  ···-·kubectl·-n·"$KUBE_NAMESPACE"·rollout·status·deployment/containerapp␣
```

**Vielen Dank für
Ihre Aufmerksamkeit**



ORDIX AG
Aktiengesellschaft für Softwareentwicklung,
Schulung, Beratung und Systemintegration

Seminarzentrum Wiesbaden
Kreuzberger Ring 13
65205 Wiesbaden
Tel.: 0611 77840-00
Fax: 0180 1 67349 0

seminare@ordix.de
<https://www.seminare.ordix.de/>